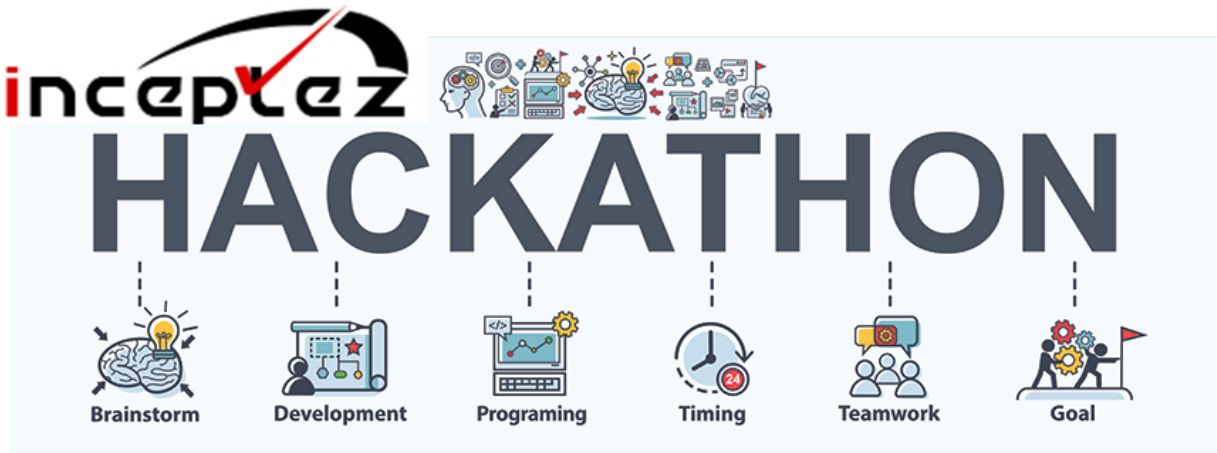



*Its time for the Challenge ..... Let us Experiment, Explore, Evolve & Excel It !!!*

# Spark (Core, SQL) Hackathon based on Interview scenarios

## Hackathon Components: Spark Core, Spark DF DSL/SQL

***We got a very good response, outcome and feedback from our students after completing these usecases such as it improved self learning capabilities, interview attending confidence, improved technical vibration, grip in coding, implementation of end to end pipeline etc., Kindly consider all the above factors and start work on it applying time and efforts fully by re trying several times until get the results.***

INCEPTEZ TECHNOLOGIES

***This Hackathon helps you to manage the cleansing, scrubbing, curation, cleanup, sanitization, preprocessing, regulation, transformation, ETL/ELT, Performance tuning and schema migration of unpredicted data sets using Spark core, SQL, Dataframe functions which can help you solving interview questions by splitting, merging, applying functions in data sets. It carries lot of important interview questions also..***

**Please read the below points before proceeding to the Hackathon:**

- **Download the data (custs_states, insuranceinfo1, insuranceinfo2 ) into HDFS location (/user/hduser/sparkhack2) and start progress.**
- **Import required classes including sparkcontext, sqlcontext and other required objects, classes and import other required libraries later as and when it is needed.**
- **Create Sparkcontext, sqlcontext or spark session.**
- **Provide the final code developed in Pycharm or you can use REPL for snippet development, but finally code should be in Pycharm.**

- **Check where ever performance can be improved and add accordingly.**
- **Follow the hints if you struck anywhere for syntax or similar examples, refer pdfs, programs, online, worst case seek for help from others. If you are struck for long time in some steps ignore those steps or work on other usecases then come back and try.**
- **All the use cases given below are categorized and stated with completion %, try to achieve maximum percentage of 100 by completing all scenarios.**
- **Provide the results as code and snapshot of results.**

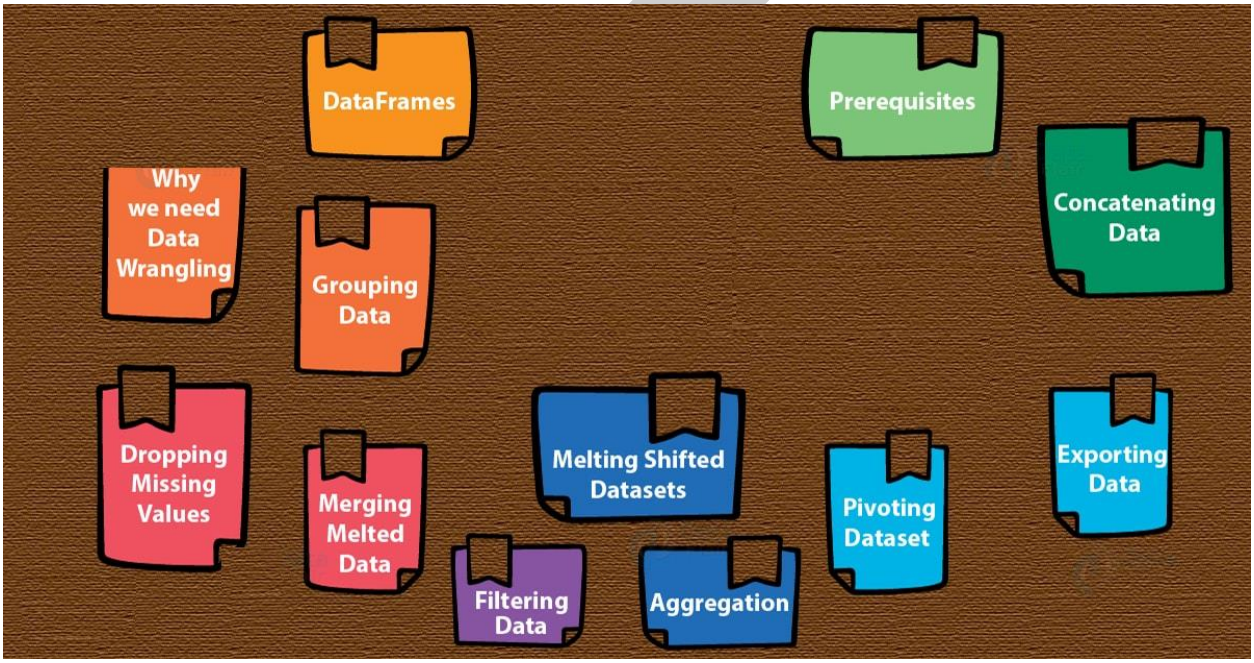
**Note: Give more priority to SQL then Core/Streaming. Minimum SQL you try to complete and send.**

**First start with Part A - Step 1 and 2 (rdd)**

**or**

**First start with Part B - Step 3, 4 and 5 (dsl/sql)**

**… as per your convenience, try to complete all - it's a matter of time and continuous efforts.**



# Part A - SPARK CORE RDD TRANSFORMATIONS / ACTIONS (Step 1 & 2)

## 1. Data cleaning, cleansing, scrubbing (20% Completion)

### Loading RDDs, remove header, blank lines and impose caseclass schema

1. **Load** the file1 (insuranceinfo1.csv) from HDFS using textFile API into an RDD insuredata
2. **Remove the header** line from the RDD contains column names.

   **Hint:**

   Use first and filter functions to achieve this by filtering the first line identified .
3. **Remove the Footer/trailer** also which contains "footer,402,,,,,,,,"

   Hint: Use functions like zipWithIndex, count and filter the last value to remove the footer or use just filter to remove if you can't implement in the above way.
4. Display the **count** and **show** few rows and check whether header and footer is removed.
5. **Remove the blank lines** in the rdd.

   **Hint:** Before splitting, trim it, calculate the length and filter only row size is not = 0.
6. **Map and split** using ',' delimiter.

   **Hint:** To avoid getting incomplete rows truncated, use **split(",",-1)** instead of split(",")

   **For Eg:**

   If you take the rdd.first it should show the output like the below

   ['84966', '84966', '2019-10-01', 'SC', 'HIOS', 'Family Enhanced Dental Plan (High)', 'https://www.metlife.com/dental', '13', '', '']

   Output should not be like given below

   ['84966', '84966', '2019-10-01', 'SC', 'HIOS', 'Family Enhanced Dental Plan (High)', 'https://www.metlife.com/dental', '13']
7. **Filter number of fields** are equal to 10 columns only - analyze why we are doing this and provide your view here..
8. **Take the count of the RDD** created in step 7 and step 1 and print how many rows are removed/rejected in the cleanup process of removing the number of fields does not equals 10.
9. **Create another RDD namely rejectdata** and store the row that does not equals 10 columns. With a new column added in the first column called numcols contains number of columns in the given row **For eg - the output should be like this:**

INCEPTEZ TECHNOLOGIES

**[(6, ['219', '219', '2019-10-01', 'AK', 'HIOS', 'ODS Premier'])] where 6 is the number of columns and ['219', '219', '2019-10-01', 'AK', 'HIOS', 'ODS Premier'] is the data, hence we can analyse the IssuerIds contains deficient fields.**

10. **Load** the file2 (insuranceinfo2.csv) from HDFS using textFile API into an RDD insuredata2
11. **Repeat from step 2 to 8 for this file** also and create the schema rdd from the insuranceinfo2.csv and **filter the records that contains blank or null IssuerId,IssuerId2 for eg: remove the records with pattern given below.**

    ,,,,,,,13,Individual,Yes

    with the field names used as per the header record in the file and apply to the above data to create schemaed RDD

## 2. Data merging, Deduplication, Performance Tuning & Persistance (15% Completion) – Total 35%

12. **Merge** the both header and footer removed RDDs derived in steps 8 and 11 into an RDD namely insuredatamerged
13. **Persist** the step 12 RDD to memory by serializing.
14. **Calculate the count of rdds** created in step 8+11 and rdd in step 12, check whether they are matching.
15. **Increase the number of partitions in the above rdd to 8 partitions** and name it as insuredatarepart.
16. **Split the above RDD using the businessdate field** into rdd_20191001 and rdd_ 20191002 based on the BusinessDate of 2019-10-01 and 2019-10-02 respectively using **Filter** function.
17. **Store the RDDs** created in step 15, 16 into HDFS location.
18. **Convert the RDDs created in step 15 above into Dataframe** namely insuredaterepartdf using

    rdd.toDF().toDF("IssuerId","IssuerId2","BusinessDate","StateCode","SourceName","NetworkName","NetworkURL","custnum","MarketCoverage","DentalOnlyPlan") function

INCEPTEZ TECHNOLOGIES

# Part B - Spark DF & SQL (Step 3, 4 & 5)

## 3. DataFrames operations (20% Completion) – Total 55%

**Apply Structure, DSL column management functions, transformation, custom udf & schema migration.**

19. Dataframe creation using the built in modules

    **A.** Create first structuretypes for all the columns as per the **insuranceinfo1**.csv with the columns such as IssuerId,IssuerId2,BusinessDate,StateCode,SourceName,NetworkName,NetworkURL,custnum,MarketCoverage,DentalOnlyPlan

    **Hint:** Do it carefully without making typo mistakes. Fields issuerid, issuerid2 should be of IntegerType, ***businessDate should be DateType*** and all other fields are StringType.

    **B.** Create second structuretypes for all the columns as per the **insuranceinfo2**.csv with the columns such as IssuerId,IssuerId2,BusinessDate,StateCode,SourceName,NetworkName,NetworkURL,custnum,MarketCoverage,DentalOnlyPlan

    **Hint:** Do it carefully without making typo mistakes. Fields issuerid, issuerid2 should be of IntegerType, ***businessDate should be StringType*** and all other fields are StringType.

    **C.** Create third structuretypes for all the columns as per the **insuranceinfo2**.csv with the columns such as IssuerId,IssuerId2,BusinessDate,StateCode,SourceName,NetworkName,NetworkURL,custnum,MarketCoverage,DentalOnlyPlan**,RejectRows**

    **Hint:** Do it carefully without making typo mistakes. Fields issuerid, issuerid2 should be of IntegerType, ***businessDate should be StringType*** and all other fields are StringType.

20. Create dataframe using the csv module accessing the insuranceinfo1.csv file and remove the footer from both using header true and remove the footer using dropmalformed options and apply the schema of the structure type created in the step 19.A

INCEPTEZ TECHNOLOGIES

Create another dataframe using the csv accessing the insuranceinfo2.csv file and remove the header from both dataframes using header true and remove the footer using dropmalformed options and apply the schema of the structure type created in the step 19.B

**Note:**for insuranceinfo2.csv as the BusinessDate column format is dd-MM-yyyy, we need to explicitly convert it to yyyy-MM-dd format using to_date() function.

Create another dataframe using the csv accessing the insuranceinfo2.csv file and remove the header from the dataframe using header true, permissive options and apply the schema of the structure type created in the step 19.C with the csv options of

I. columnnameofcorruptrecord and store the rejected data in a new DF.

Eg: ,,,,,,,13,SHOP (Small Group) this record has to be rejected

II. Ignoreleading and trailing whitespaces

,,,,,,,13, SHOP (Small Group) ,Yes

Finally store the rejected records in a new dataframe in a single csv file.

LINCEHEZ TECHNOLOGIES

21. Apply the below **DSL** functions in the DFs created in step 20.
    a. **Rename** the fields StateCode and SourceName as stcd and srcnm respectively.
    b. **Concat** IssuerId,IssuerId2 as issueridcomposite and make it as a new field

       **Hint :** Cast to string and concat.
    c. **Remove** DentalOnlyPlan column
    d. **Add columns** that should show the current system date and timestamp with the fields name of sysdt and systs respectively.

**Try the below interesting usecases seperately:**

i. Identify all the column names and store in an List variable – use **columns** function.

ii. Identify all columns with datatype and store in a list variable and print – use **dtypes** function.

iii. Identify all integer columns alone and store in an list variable and print.

iv. Select only the integer columns identified in the above statement and show 10 records in the screen.
Hint: add all the column names in a list and apply it in df.select(list)

v. Identify the additional column in the reject dataframe created in step 20 above by subtracting the columns between dataframe1 and dataframe3 created in step20. Hint: use columns function, then convert into set type and then do a subtraction.

22. **Take the DF created in step 21.d and Remove the rows** contains null in any one of the field and count the number of rows which contains all columns with some value.

23. **Custom Method creation:** Create a package (org.inceptez.hack), module (allmethods), method (remspecialchar)
**Hint:** First create the function/method directly and then later add in a seperate pkg.module
    a. Method should take 1 string argument and 1 return of type string
    b. Method should remove all special characters and numbers 0 to 9 - ? , / _ ( ) [ ]
    **Hint:** Use python regular expression function ie "re" function, usage of [] symbol should use \\ escape sequence.
    Eg: regexp = re.sub("[;\\@%-^/:~,*?\"<>|&'0-9]",'',a)
    **c.** For eg. If I pass to the above method value as **Pathway - 2X (with dental)** it has to return **Pathway X with dental** as output.

INCEPTEZ TECHNOLOGIES

24. Import the package and refer the method generated in step 23 as a udf for invoking in the DSL function.
25. Call the above udf in the DSL by passing NetworkName column as an argument to get the special characters removed DF.
26. Save the DF generated in step 25 in JSON format into HDFS with overwrite option.
27. Save the DF generated in step 25 into CSV format with header name as per the DF and delimited by ~ into HDFS with overwrite option.

Note: Create a generic function namely writeToFile that should have 5 arguments passed as sparksession, filetype, location, delimiter, mode and call this function in the above step 26 and 27 to save the data rather than calling the write.csv and write.json directly.

28. Save the DF generated in step 25 into hive external table and append the data without overwriting it.

## 4. Tale of handling RDDs, DFs and TempViews (20% Completion) – Total 75%

**Loading RDDs, split RDDs, Load DFs, Split DFs, Load Views, Split Views, write UDF, register to use in Spark SQL, Transform, Aggregate, store in disk/DB**

**Use RDD functions:**

29. **Load** the file3 (custs_states.csv) from the HDFS location, using textfile API in an RDD custstates, this file contains 2 type of data one with 5 columns contains customer master info and other data with statecode and description of 2 columns.
30. **Split** the above data into 2 RDDs, first RDD namely custfilter should be loaded only with 5 columns data and second RDD namely statesfilter should be only loaded with 2 columns data.

INCEPTEZ TECHNOLOGIES

**Use DSL functions:**

31. **Load** the file3 (custs_states.csv) from the HDFS location, using CSV Module in a DF custstatesdf, this file contains 2 type of data one with 5 columns contains customer master info and other data with statecode and description of 2 columns.
32. **Split** the above data into 2 DFs, first DF namely custfilterdf should be loaded only with 5 columns data and second DF namely statesfilterdf should be only loaded with 2 columns data.

    **Hint:** Use filter/where DSL function to check isnull or isnotnull to achieve the above functionality then rename, change the type and drop columns in the above 2 DFs accordingly.

**Use SQL Queries:**

33. **Register the above step 32 two DFs as temporary views** as custview and statesview.
34. **Register the DF generated in step 21.d as a tempview** namely insureview
35. **Import the package and refer the method** created in step 23 in the name of remspecialcharudf using **spark udf registration**.
36. Write an **SQL query** with the below processing – set the spark.sql.shuffle.partitions to 4
    a. Pass NetworkName to remspecialcharudf and get the new column called cleannetworkname
    b. Add current date, current timestamp fields as curdt and curts.
    c. Extract the year and month from the businessdate field and get it as 2 new fields called yr,mth respectively.
    d. Extract from the protocol either http/https from the NetworkURL column, if http then print http non secured if https then secured else no protocol found then display **noprotocol.** For Eg: if http://www2.dentemax.com/ then show **http non secured** else if https://www2.dentemax.com/ then **http secured** else if www.bridgespanhealth.com then show as **no protocol** store in a column called protocol.
    e. Display all the columns from insureview including the columns derived from above a, b, c, d steps with statedesc column from statesview with age,profession column from custview . Do an Inner Join of insureview with statesview using stcd=stated and join insureview with custview using custnum=custid.
37. Store the above selected Dataframe in **Parquet** formats in a HDFS location as a **single file**.
38. Write an SQL query to identify average age, count group by statedesc, protocol, profession including a seqno column added which should have running sequence number partitioned based on protocol and ordered based on count descending and display the profession whose second highest **count** of a given state and protocol.
    For eg.
    **Seqno**,Avgage,count,statedesc,protocol,profession

INCEPTEZ TECHNOLOGIES

**1**,48.4,10000, Alabama,http,Pilot

**2**,72.3,300, Colorado,http,Economist

**1**,48.4,3000, Atlanta,https,Health worker

**2**,72.3,2000, New Jersey,https,Economist

39. Store the DF generated in step 38 into MYSQL table insureaggregated.
40. *Test the code in REPL/PyCharm and save it as a .py file and try submit with driver memory of 512M, number of executors as 4, executor memory as 1GB and executor cores with 2 cores.*

## 5. Visualization (5% Completion) – Total 80%

Login to spark ui and take the snapshot of the below items.

1. Jobs
2. Stages

   **Display the below items**

   a. Task locality level in the Tasks
   b. Scheduler Delay
   c. Task Deserialization Time
   d. Shuffle Read Blocked Time
   e. Shuffle Remote Reads
   f. Result Serialization Time
   g. Getting Result Time
   h. Peak Execution Memory
3. Storage
   a. Storage level
   b. Partitions info
4. Executors info.
5. SQL
   a. DAG
   b. AST (Abstract Syntax Tree plan)

INCEPTEZ TECHNOLOGIES

## Additional (Must try) Hackathon Usecases to explore:

Try the below one of both usecases as well:

**Usecase1:** Complex data structurization and standardization to achieve Schema Migration

1. Make use of the structtype as mentioned below.

```
from pyspark.sql.types import *
schema = StructType([
        StructField("results", ArrayType(StructType([
                    StructField("gender", StringType()),
                    StructField("location",
                            StructType([StructField("city",
StringType()),StructField("state", StringType()),StructField("country",
StringType()),StructField("postcode", StringType()),
                            StructField("coordinates",
StructType([StructField("latitude", StringType()),StructField("longitude",
StringType())])),])),
                    StructField("name", StructType([StructField("title",
StringType()),StructField("first", StringType()),StructField("last",
StringType())])),
                    StructField("email", StringType()),
                    StructField("login", StructType([StructField("uuid",
StringType()),StructField("username", StringType())])),
                    StructField("dob", StructType([StructField("date",
StringType()),StructField("age", StringType())])),
                    StructField("registered",
StructType([StructField("date", StringType()),StructField("age",
StringType())])),

        ])))])
```

2. Read the complex json file provided (apidata.json) as a DF by applying the structure type provided above.
3. Print the schema of the above DF created with multiline option true.
4. Structurize the complex json using explode function to flatten the array to rows as given below.

```
exploded_df = jsondf.selectExpr('explode(results) as exploded_cols')
flattened_df = exploded_df \
        .withColumn('gender', col('exploded_cols.gender')) \
        .withColumn('city',col('exploded_cols.location.city'))\
        .withColumn('state',col('exploded_cols.location.state'))\
        .withColumn('title', col('exploded_cols.name.title')) \
        .withColumn('first',col('exploded_cols.name.first'))\
        .withColumn('last',col('exploded_cols.name.last'))\
```

INTERTECHNOLOGIES

```
.withColumn('latitude',col('exploded_cols.location.coordinates.latitude'))\

.withColumn('longitude',col('exploded_cols.location.coordinates.longitude'))\
        .withColumn('email', col('exploded_cols.email')) \
        .withColumn('uuid',col('exploded_cols.login.uuid'))\
        .withColumn('username',col('exploded_cols.login.username'))\
        .withColumn('dob_dt',col('exploded_cols.dob.date'))\
        .withColumn('dob_age',col('exploded_cols.dob.age'))\
        .withColumn('login_dt', col('exploded_cols.registered.date')) \
        .withColumn('login_age',col('exploded_cols.registered.age'))\
        .drop('exploded_cols')
```

5. Identify the below results out of it using DSL and SQL
   a. Identify the gender wise count
   b. Identify the gender wise average age
   c. Extract only the date portion of the dob column
   d. Apply md5 masking algorithm to the password column
   e. Store the cell number in the format of 000-000-0000 for eg. 8256249837 has to 825-624-9837
   f. Calculate the age is calculated correctly using the dob compared with the age column
   g. Identify the email is valid

ACEPTEZ TECHNOLOGIES

**<u>Usecase2:</u>**

**<u>Manufacturing Data Processing Usecase -</u>** Data Munging, Enrichment and Wrangling functinalities.

**Requirement Spec-** Manufacturing data shall be prepared so it can be used in a machine-learning model. As part of the proof-of-concept, your person-of-contact in the plant provides you with about 1h worth of data collected at a small production line. The data-scientists building the machine-learning model have requested that you provide it to them in the "tidy" format.

***The Tasks:***

1. Please familiarize yourself with the given data set (manuf_data.csv) and take notes of any peculiarities you discover throughout working on the dataset.

2. Implement Data munging, Enrichment and Wrangling logics using pyspark DSL/SQL.

INCENTEZ TECHNOLOGIES

3. Provide your application logic as a script that can be called from the command line. This script shall be deployed and executed in a production environment

4. Submit the tidy data table in CSV format alongside of your solution's source code

5. Create a presentation as like the given ppt (Manufacture data Code Challenge solution.pptx) mentioning the data pipeline strategies used and send it along with the hackathon usecases code.